

选择题

1. 设二维矩阵 $f = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$, $h = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, 则其卷积 $g = f * h$ 的尺寸为_____，矩阵 g 右下角元素值为_____ **B** 。
- A. 3×3 , 36 B. 4×4 , 36 C. 3×3 , 63 D. 4×4 , 63

2. 在下述关于灰度级变换的论述中，错误的是_____ **C** 。
- A. 灰度级变换通常可以使用查表的方法实现
- B. 直方图均衡化是一种灰度级变换方法
- C. 灰度级变换与像素位置相关，在改变像素的亮度时要考虑该像素原来的亮度及其在图像中的位置
- D. 灰度级变换将原来在范围 $[p_0, p_k]$ 内的亮度 p 变换为一个新范围 $[q_0, q_k]$ 内的亮度 q

3. 迭代的（最优）阈值选择算法是一种简单的阈值选择算法，该算法利用_____ **A** 设定阈值。
- A. 类内平均灰度值
- B. 直方图两个局部极大值之间的极小值处的灰度值
- C. 最大化类间方差的灰度值
- D. 局部阈值化方法

4. 下图 1 是一幅二值图像，使用直径为 20 的圆形结构元素对该图像进行二值形态学运算，形成的结果图像如图 2 所示，这是由_____ **B** 处理得到的？



- A. 开运算 B. 闭运算 C. 膨胀 D. 腐蚀

5. SIFT 特征检测利用_____ **B** 设定特征点的方向。
- A. 特征向量归一化
- B. 邻域梯度方向直方图
- C. 坐标轴旋转
- D. 高斯尺度空间极值

判断题

- 1. 在数字图像中，彩色图像通过矢量函数进行描述。(√)
- 2. 在边缘检测中，可先对原始图像进行平滑滤波，进而获取图像的二阶微分最大值以检测边缘。(×)
- 3. Harris 算子对图像缩放具有不变性。(×)
- 4. 利用对极约束可以将对应点搜索空间的维度由二维降为一维。(√)

简答

1. 对于如下有 10 个亮度级、大小为 8×8 的图像 I，试给出该图像的直方图。

$$I = \begin{bmatrix} 0 & 3 & 5 & 6 & 1 & 0 & 2 & 7 \\ 3 & 3 & 3 & 1 & 1 & 1 & 2 & 6 \\ 9 & 3 & 0 & 0 & 1 & 9 & 7 & 7 \\ 9 & 2 & 0 & 0 & 4 & 9 & 0 & 6 \\ 2 & 2 & 2 & 4 & 4 & 4 & 2 & 7 \\ 0 & 2 & 7 & 8 & 4 & 0 & 2 & 0 \\ 8 & 5 & 1 & 8 & 1 & 5 & 1 & 7 \\ 9 & 8 & 5 & 9 & 5 & 5 & 1 & 7 \end{bmatrix}$$

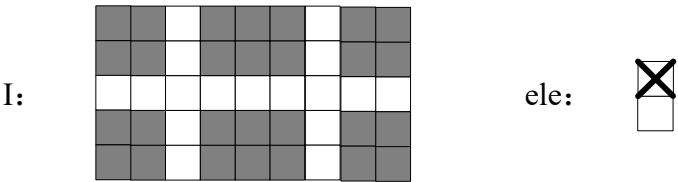
解答：

直方图为：

0	1	2	3	4	5	6	7	8	9
10	9	9	5	5	6	3	7	4	6

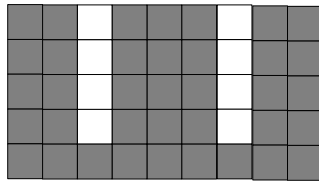
(表格画图均可)

2. 对于如下二值图像 I (白色表示 1，灰色表示 0)，若结构元素为 ele，试给出图像进行开运算的结果。

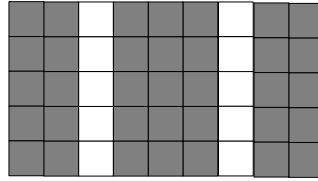


解答：

开运算是先腐蚀后膨胀。

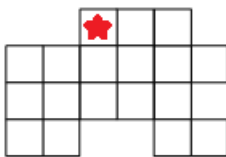


对 I 进行腐蚀后的结果为：



再进行膨胀后的结果为：

3. 对如下二值图像进行 8-邻接内边界跟踪，请在图中画出跟踪结果，并用虚线表示测试过的方向。



（起点为标志位置，按照算法依次画出测试方向和最终结果即可）

4. 说明立体视觉的基本原理。

参考解答：空间中一点 P ，用摄像机 C_1 进行观察，其在图像中的位置为 P_1 ，设 C_1 的光心位置为 O_1 ，则可确定空间点 P 位于 O_1P_1 连线上，而深度未知。同理，用摄像机 C_2 观察同一点 P ，其在图像中的位置为 P_2 ，设 C_2 的光心位置为 O_2 ，则 P 点同时位于 O_2P_2 连线上。因此， P 点为直线 O_1P_1 和 O_2P_2 的交点，其三维位置唯一确定。

5. 简要说明 SIFT 特征检测方法具有旋转不变性的原因。

参考解答：SIFT 特征检测方法为每一个特征点计算主方向，主方向与该点邻域像素的梯度有关。当图像发生旋转时，特征点的主方向相对于像素的梯度方向不变，将待匹配的图像旋转到令特征点方向为 0 的位置再匹配，从而使特征具有旋转不变性。

解答

1. 对于如下有 8 个亮度级、大小为 6×6 的图像，试给出对该图像进行直方图均衡化的结果。

$$I = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 & 4 \\ 0 & 0 & 2 & 1 & 3 & 4 \\ 3 & 0 & 4 & 7 & 1 & 5 \\ 2 & 1 & 6 & 4 & 4 & 4 \\ 0 & 1 & 3 & 7 & 0 & 6 \\ 3 & 0 & 7 & 4 & 2 & 2 \end{bmatrix}$$

解答：

图像 I 的直方图 H 为：

0	1	2	3	4	5	6	7
7	6	5	5	7	1	2	3

累计直方图 H_c 为：

0	1	2	3	4	5	6	7
7	13	18	23	30	31	33	36

根据直方图均衡化公式：

$$T[p] = \text{round}\left(\frac{k-1}{MN} H_c[p]\right), p = 0, 1, \dots, k-1 \quad (k=8)$$

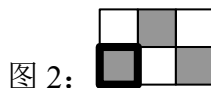
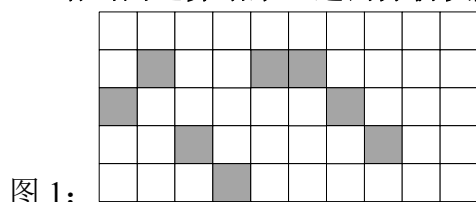
得到查找表 T 为：

0	1	2	3	4	5	6	7
1	3	4	4	6	6	6	7

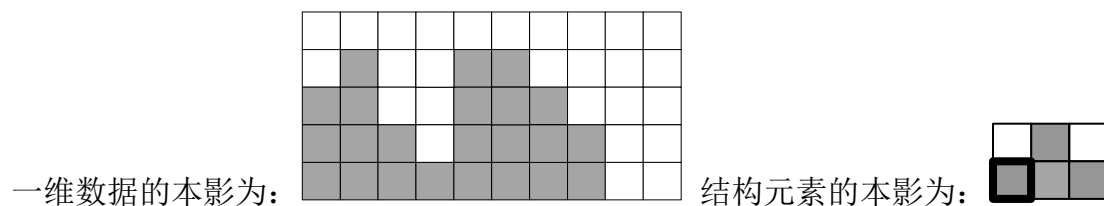
因此，直方图均衡化以后的图像为：

$$I = \begin{bmatrix} 3 & 4 & 1 & 3 & 4 & 6 \\ 1 & 1 & 4 & 3 & 4 & 6 \\ 4 & 1 & 6 & 7 & 3 & 6 \\ 4 & 3 & 6 & 6 & 6 & 6 \\ 1 & 3 & 4 & 7 & 1 & 6 \\ 4 & 1 & 7 & 6 & 4 & 4 \end{bmatrix}$$

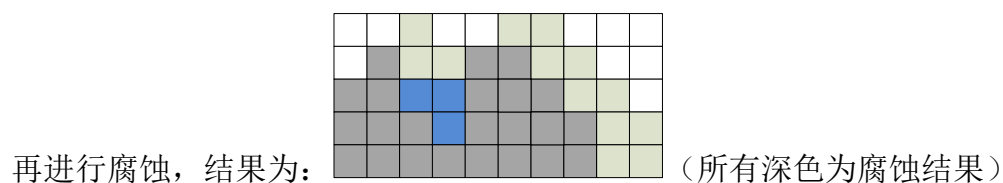
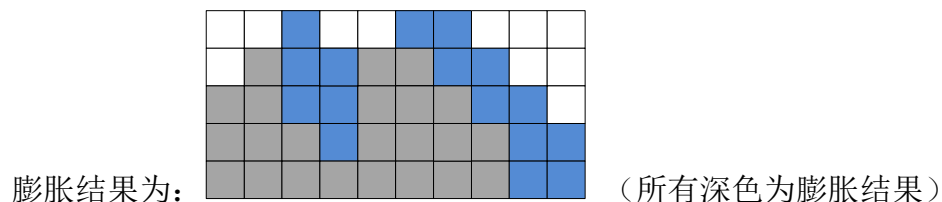
2. 对于图 1 所示的一维数据，使用图 2 所示的结构元素进行灰度级闭运算，试给出闭运算结果，进而分析灰度级闭运算的作用。



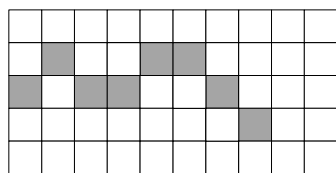
解答:



将两者作为二维二值图像进行闭运算（先膨胀再腐蚀）



该图像的顶面即为原始一维数据进行灰度开运算的结果:



将闭运算结果与原图相比发现，灰度级闭运算可以去除比结构元素小的暗色细节，同时保持灰度级不变。

（使用公式计算也可以）

3. 设计程序实现 Harris 角点检测算法中的非极大值抑制，函数接口形式为：

Re=notMax(I, m, n, size)

其中，**I** 是待处理矩阵，**Re** 是结果，**m** 和 **n** 分别为矩阵 **I** 的长和宽，**size** 是非极大值抑制的邻域窗口大小。编程语言可选择 C/C++或 matlab。

参考程序：

Matlab 代码如下：

```
function re = notMax(I, m, n, size)
re = zeros(m,n);
r = floor(size/2);
for i=1+r:m-r
    for j=1+r:n-r
        imax = -1;
        for k=-r:r
            for l=-r:r
                if I(i+k,j+l)>imax
                    imax = I(i+k,j+l);
                end
            end
        end
        if I(i,j) == imax
            re(i,j) = imax;
        end
    end
end
```

4. 编程实现 SIFT 特征匹配函数，定义函数接口如下：

re=SIFTMatch(descriptor, database, n, th);

其中，**descriptor** 是待匹配的 SIFT 特征向量，其数据类型是 **double** 型一维数组 (C/C++)或 **double** 型向量(Matlab); **database** 是存储特征向量的二维数组(C/C++)或矩阵(Matlab)，特征向量的数目为 **n**; **th** 是特征匹配过程中所需阈值; **re** 是匹配结果，匹配成功返回成功匹配的特征向量编号，否则返回-1。编程语言可选择 C/C++或 Matlab。

参考程序

Matlab代码如下：

```
function re = SIFTMatch(descriptor, database, n, th)
min = 10000000000;
secmin = 10000000000;
minNo = 0;
for i=1:n
    vec = database(i,:);
    dis = sqrt(sum((descriptor-vec).^2)); % 计算欧式距离
    if dis < min
        secmin = min;
        min = dis;
        minNo = i;
    else if dis < secmin
        secmin = dis;
    end
end
end
if min/secmin < th
    re = minNo;
else
    re = -1;
end
```